

LAB TEST-03

Name : P.AJAY

Rollno :2403A510B4

Batch :05

Course :AI Assisted Coding

Set-E1

Q1:

Scenario: In the Hospitality sector, a company faces a challenge related to code refactoring.

Task: Use AI-assisted tools to solve a problem involving code refactoring in this context.

Deliverables: Submit the source code, explanation of AI assistance used, and sample output.

Prompt:# use AI-assisted tools to solve a problem involving code refactoring in this context.

Code :

```

1  """
2  Refactored Hospitality Data Processor
3
4  - Cleans reservation and room-inventory CSVs
5  - Handles missing/invalid values
6  - Detects overbookings and anomalous revenue entries
7  - Aggregates occupancy and revenue by room type and date
8  - Small demo when run directly
9  """
10 from __future__ import annotations
11 from dataclasses import dataclass
12 from typing import List, Dict, Optional, Tuple
13 import csv
14 import statistics
15 import pathlib
16 import random
17 from datetime import datetime, timedelta
18
19
20 @dataclass
21 class Reservation:
22     reservation_id: str
23     hotel_id: str
24     guest_name: str
25     room_type: str
26     check_in: Optional[datetime]
27     check_out: Optional[datetime]
28     price_per_night: float
29     nights: int
30     total_price: float
31     status: str # e.g., booked, checked_out, cancelled
32
33
34 class HotelProcessor:
35     """Processor for hotel reservations and room inventory."""
36
37     def __init__(self, reservations: Optional[List[Reservation]] = None, capacity_by_type: Optional[Dict[str, int]] = None):

```

```

38         self.reservations: List[Reservation] = reservations or []
39         # capacity_by_type maps room_type -> number of rooms
40         self.capacity_by_type: Dict[str, int] = capacity_by_type or {}
41
42     @staticmethod
43     def from_csv(path: str, capacity_by_type: Optional[Dict[str, int]] = None) -> "HotelProcessor":
44         """Load CSV with headers: reservation_id,hotel_id,guest_name,room_type,check_in,check_out,price_per_night,status"""
45         reservations: List[Reservation] = []
46         with open(path, newline="", encoding="utf-8") as f:
47             reader = csv.DictReader(f)
48             for r in reader:
49                 try:
50                     ci = HotelProcessor._parse_dt(r.get("check_in"))
51                     co = HotelProcessor._parse_dt(r.get("check_out"))
52                     price = float(r.get("price_per_night") or 0.0)
53                     nights = HotelProcessor._compute_nights(ci, co)
54                     total = float(r.get("total_price") or (price * nights))
55                     reservations.append(
56                         Reservation(
57                             reservation_id=str(r.get("reservation_id", "")).strip(),
58                             hotel_id=str(r.get("hotel_id", "")).strip(),
59                             guest_name=str(r.get("guest_name", "")).strip(),
60                             room_type=str(r.get("room_type", "standard")).strip(),
61                             check_in=ci,
62                             check_out=co,
63                             price_per_night=price,
64                             nights=nights,
65                             total_price=total,
66                             status=str(r.get("status", "booked")).strip(),
67                         )
68                 )
69             except Exception:
70                 continue
71         return HotelProcessor(reservations, capacity_by_type)
72

```

```

74 def _parse_dt(val: Optional[str]) -> Optional[datetime]:
75     if not val or str(val).strip() == "":
76         return None
77     for fmt in ("%Y-%m-%d", "%Y-%m-%d %H:%M:%S", "%d/%m/%Y"):
78         try:
79             return datetime.strptime(val, fmt)
80         except Exception:
81             continue
82     try:
83         # fallback to ISO parse
84         return datetime.fromisoformat(val)
85     except Exception:
86         return None
87
88 @staticmethod
89 def _compute_nights(ci: Optional[datetime], co: Optional[datetime]) -> int:
90     if not ci or not co:
91         return 0
92     delta = (co.date() - ci.date()).days
93     return max(0, delta)
94
95 @staticmethod
96 def sample_data(n: int = 40) -> "HotelProcessor":
97     """Generate sample reservations and a small capacity map for demo/testing."""
98     room_types = ["standard", "deluxe", "suite"]
99     cap = {"standard": 10, "deluxe": 5, "suite": 2}
100     recs: List[Reservation] = []
101     base_date = datetime(2025, 6, 1)
102     for i in range(n):
103         rt = random.choice(room_types)
104         start = base_date + timedelta(days=random.randint(0, 14))
105         nights = random.randint(1, 5)
106         end = start + timedelta(days=nights)
107         price = {"standard": 80.0, "deluxe": 150.0, "suite": 300.0}[rt] * random.uniform(0.8, 1.2)
108         # inject occasional missing price or bad dates
109         if random.random() < 0.05:

```

```

110         price = 0.0
111         if random.random() < 0.03:
112             end = start # zero nights (invalid)
113             total = round(price * nights, 2)
114             recs.append(
115                 Reservation(
116                     reservation_id=f"R{i+1}",
117                     hotel_id="H1",
118                     guest_name=f"Guest{i+1}",
119                     room_type=rt,
120                     check_in=start,
121                     check_out=end,
122                     price_per_night=round(price, 2),
123                     nights=nights,
124                     total_price=total,
125                     status="booked",
126                 )
127             )
128         return HotelProcessor(recs, cap)
129
130     def clean(self) -> None:
131         """Clean reservations in-place:
132         - remove records with missing or invalid dates
133         - fill missing prices with median per room type
134         - compute nights/total_price where missing
135         - remove exact duplicates (reservation_id)
136         """
137         # drop reservations without valid check_in/check_out or zero nights
138         valid = [r for r in self.reservations if r.check_in and r.check_out and r.nights > 0]
139         self.reservations = valid
140
141         # median price per room type (ignore zero or missing)
142         prices: Dict[str, List[float]] = {}
143         for r in self.reservations:

```

```

144         if r.price_per_night and r.price_per_night > 0:
145             prices.setdefault(r.room_type, []).append(r.price_per_night)
146     medians = {rt: (statistics.median(vals) if vals else 0.0) for rt, vals in prices.items()}
147
148     # fill missing prices, recompute totals, deduplicate by reservation_id
149     seen_ids = set()
150     cleaned: List[Reservation] = []
151     for r in self.reservations:
152         if r.reservation_id in seen_ids:
153             continue
154         seen_ids.add(r.reservation_id)
155         if r.price_per_night <= 0:
156             r.price_per_night = medians.get(r.room_type, 0.0)
157         if r.nights == 0:
158             r.nights = self._compute_nights(r.check_in, r.check_out)
159         if r.total_price <= 0:
160             r.total_price = round(r.price_per_night * r.nights, 2)
161         cleaned.append(r)
162     self.reservations = cleaned
163
164     def occupancy_by_date_and_type(self) -> Dict[Tuple[datetime.date, str], int]:
165         """Compute number of occupied rooms for each date and room_type."""
166         occ: Dict[Tuple[datetime.date, str], int] = {}
167         for r in self.reservations:
168             for d in range(r.nights):
169                 day = (r.check_in + timedelta(days=d)).date()
170                 k = (day, r.room_type)
171                 occ[k] = occ.get(k, 0) + 1
172         return occ
173
174     def detect_overbookings(self) -> List[Tuple[datetime.date, str, int, int]]:
175         """Return list of (date, room_type, occupied, capacity) where occupied > capacity."""
176         issues = []
177         occ = self.occupancy_by_date_and_type()

```



```

178         for (day, rt), occupied in occ.items():
179             cap = self.capacity_by_type.get(rt, 0)
180             if cap and occupied > cap:
181                 issues.append((day, rt, occupied, cap))
182         return issues
183
184     def detect_revenue_anomalies(self, z_threshold: float = 3.0) -> List[Reservation]:
185         """Detect reservations where total_price per night is an outlier for the room type."""
186         per_night = {}
187         for r in self.reservations:
188             if r.nights > 0:
189                 v = round(r.total_price / r.nights, 2)
190                 per_night.setdefault(r.room_type, []).append((r, v))
191         anomalies: List[Reservation] = []
192         for rt, items in per_night.items():
193             vals = [v for (_, v) in items]
194             if len(vals) < 2:
195                 continue
196             mean = statistics.mean(vals)
197             stdev = statistics.pstdev(vals) or 0.0
198             for r, v in items:
199                 z = 0.0 if stdev == 0 else abs((v - mean) / stdev)
200                 if z > z_threshold:
201                     anomalies.append(r)
202         return anomalies
203
204     def aggregate_by_room_type(self) -> Dict[str, Dict[str, float]]:
205         """Aggregate occupancy nights and revenue by room type."""
206         agg: Dict[str, Dict[str, float]] = {}
207         for r in self.reservations:
208             a = agg.setdefault(r.room_type, {"nights": 0.0, "revenue": 0.0, "bookings": 0})
209             a["nights"] += r.nights
210             a["revenue"] += r.total_price
211             a["bookings"] += 1
212         for rt, v in agg.items():

```

```

213             v["rev_per_night"] = v["revenue"] / v["nights"] if v["nights"] else 0.0
214         return agg
215
216     def summary(self) -> str:
217         agg = self.aggregate_by_room_type()
218         lines = [f"Reservations: {len(self.reservations)}"]
219         for rt, v in agg.items():
220             lines.append(
221                 f"{rt}: bookings={v['bookings']}, nights={v['nights']:.0f}, revenue={v['revenue']:.2f}, rev/night={v['rev_per_night']:.2f}"
222             )
223         over = self.detect_overbookings()
224         lines.append(f"Overbookings: {len(over)}")
225         return "\n".join(lines)
226
227     def save_csv(self, path: str) -> None:
228         p = pathlib.Path(path)
229         p.parent.mkdir(parents=True, exist_ok=True)
230         with open(p, "w", newline="", encoding="utf-8") as f:
231             writer = csv.writer(f)
232             writer.writerow(
233                 [
234                     "reservation_id",
235                     "hotel_id",
236                     "guest_name",
237                     "room_type",
238                     "check_in",
239                     "check_out",
240                     "price_per_night",
241                     "nights",
242                     "total_price",
243                     "status",
244                 ]
245             )
246         for r in self.reservations:
247             writer.writerow(

```

```

248         [
249             r.reservation_id,
250             r.hotel_id,
251             r.guest_name,
252             r.room_type,
253             r.check_in.isoformat() if r.check_in else "",
254             r.check_out.isoformat() if r.check_out else "",
255             f"{r.price_per_night:.2f}",
256             r.nights,
257             f"{r.total_price:.2f}",
258             r.status,
259         ]
260     )
261
262
263     def demo_run():
264         proc = HotelProcessor.sample_data(60)
265         print("=== Raw sample statistics ===")
266         print(f"Raw reservations: {len(proc.reservations)}")
267         anomalies_before = proc.detect_revenue_anomalies()
268         print(f"Revenue anomalies before cleaning: {len(anomalies_before)}")
269
270         proc.clean()
271         print("\n=== After cleaning ===")
272         print(proc.summary())
273
274         anomalies_after = proc.detect_revenue_anomalies()
275         print(f"\nRevenue anomalies after cleaning: {len(anomalies_after)}")
276         overbookings = proc.detect_overbookings()
277         if overbookings:
278             print("\nExample overbooking:", overbookings[0])
279
280         proc.save_csv("output/cleaned_hotel_reservations.csv")
281         print("\nSaved cleaned CSV -> output/cleaned_hotel_reservations.csv")

```

```

282
283
284     if __name__ == "__main__":
285         demo_run()

```

Output :

```

PS C:\Users\Reshm\Desktop\AIAC> & C:/Users/Reshm/anaconda3/python.exe c:/Users/Reshm/Desktop/AIAC/test-03.py
=== Raw sample statistics ===
Raw reservations: 60
Revenue anomalies before cleaning: 1

=== After cleaning ===
Reservations: 60
suite: bookings=20, nights=69, revenue=21250.85, rev/night=307.98
deluxe: bookings=22, nights=65, revenue=9682.62, rev/night=148.96
standard: bookings=18, nights=43, revenue=3434.83, rev/night=79.88
Overbookings: 16

Revenue anomalies after cleaning: 0

Example overbooking: (datetime.date(2025, 6, 11), 'suite', 7, 2)

Saved cleaned CSV -> output/cleaned_hotel_reservations.csv
PS C:\Users\Reshm\Desktop\AIAC>

```

Observation :

- Purpose: refactors hotel reservation processing — parses CSV, cleans data, detects overbookings and revenue anomalies, aggregates occupancy/revenue, and saves cleaned CSV.
- Strengths: modular, typed, handles missing prices via median fill, computes nights/totals, provides demo data and CSV I/O.
- Output: textual summary, CSV "output/cleaned_hotel_reservations.csv", and lists of anomalies/overbookings from `demo_run()`.

Q2:

Scenario: In the E-commerce sector, a company faces a challenge related to code refactoring.

Task: Use AI-assisted tools to solve a problem involving code

refactoring in this context.

Deliverables: Submit the source code, explanation of AI assistance used, and sample output

Prompt: # Use AI tools to refactor the given code to improve efficiency and readability. Provide the updated source code, explain how AI assisted the changes, and include sample output.

Code :

```
1  """
2  Refactored E-commerce Data Processor
3
4  - Cleans orders / product / customer CSVs
5  - Handles missing/invalid values
6  - Detects potential fraud/anomalies and inventory issues
7  - Aggregates sales by category and top customers
8  - Demo when run directly
9  """
10 from __future__ import annotations
11 from dataclasses import dataclass
12 from typing import List, Dict, Optional, Tuple
13 import csv
14 import statistics
15 import pathlib
16 import random
17 from datetime import datetime, timedelta
18
19
20 @dataclass
21 class Order:
22     order_id: str
23     customer_id: str
24     product_id: str
25     category: str
26     quantity: int
27     price: float
28     order_ts: Optional[datetime]
29     status: str # e.g., completed, refunded, cancelled
30     total: float
31
32
33 class ECommerceProcessor:
34     """Processor for e-commerce orders, products and customers."""
35
36     def __init__(self, orders: Optional[List[Order]] = None, product_stock: Optional[Dict[str, int]] = None):
```

```

37     self.orders: List[Order] = orders or []
38     self.product_stock: Dict[str, int] = product_stock or {}
39
40     @staticmethod
41     def _parse_dt(val: Optional[str]) -> Optional[datetime]:
42         if not val:
43             return None
44         for fmt in ("%Y-%m-%d %H:%M:%S", "%Y-%m-%d", "%d/%m/%Y %H:%M:%S"):
45             try:
46                 return datetime.strptime(val, fmt)
47             except Exception:
48                 continue
49         try:
50             return datetime.fromisoformat(val)
51         except Exception:
52             return None
53
54     @staticmethod
55     def from_csv(path: str, product_stock: Optional[Dict[str, int]] = None) -> "ECommerceProcessor":
56         """Load orders CSV with headers: order_id,customer_id,product_id,category,quantity,price,order_ts,status,total"""
57         orders: List[Order] = []
58         with open(path, newline="", encoding="utf-8") as f:
59             reader = csv.DictReader(f)
60             for r in reader:
61                 try:
62                     ts = ECommerceProcessor._parse_dt(r.get("order_ts"))
63                     qty = int(float(r.get("quantity") or 0))
64                     price = float(r.get("price") or 0.0)
65                     total = float(r.get("total") or (price * qty))
66                     orders.append(
67                         Order(
68                             order_id=str(r.get("order_id", "")).strip(),
69                             customer_id=str(r.get("customer_id", "")).strip(),
70                             product_id=str(r.get("product_id", "")).strip(),
71                             category=str(r.get("category", "uncategorized")).strip(),

```

```

72         quantity=qty,
73         price=price,
74         order_ts=ts,
75         status=str(r.get("status", "completed")).strip(),
76         total=total,
77     )
78 )
79 except Exception:
80     continue
81 return ECommerceProcessor(orders, product_stock or {})
82
83 @staticmethod
84 def sample_data(n: int = 80) -> "ECommerceProcessor":
85     """Generate sample orders and product stock for demo/testing."""
86     categories = ["electronics", "books", "apparel", "home"]
87     product_stock = {f"P{i}": random.randint(0, 50) for i in range(1, 21)}
88     orders = []
89     now = datetime(2025, 7, 1, 10, 0, 0)
90     for i in range(n):
91         pid = random.choice(list(product_stock.keys()))
92         cat = random.choice(categories)
93         cid = f"C{random.randint(1,20)}"
94         qty = random.randint(1, 5)
95         price = round(random.uniform(5, 500), 2)
96         ts = now - timedelta(minutes=random.randint(0, 60*24*30))
97         status = "completed"
98         # inject some refunds and suspicious behavior
99         if random.random() < 0.08:
100             status = "refunded"
101         # inject extreme totals
102         total = round(price * qty, 2)
103         if random.random() < 0.03:
104             total *= 10
105         orders.append(Order(f"O{i+1}", cid, pid, cat, qty, price, ts, status, total))
106     return ECommerceProcessor(orders, product_stock)

```

```

107
108 def clean(self) -> None:
109     """Clean orders in-place:
110     - remove malformed orders (missing ids, non-positive qty)
111     - fill missing totals/prices with sane defaults (median price per category)
112     - update product_stock (deduct completed orders)
113     - deduplicate by order_id
114     """
115     # drop malformed
116     self.orders = [o for o in self.orders if o.order_id and o.customer_id and o.quantity > 0]
117
118     # median price per category
119     cat_prices: Dict[str, List[float]] = {}
120     for o in self.orders:
121         if o.price > 0:
122             cat_prices.setdefault(o.category, []).append(o.price)
123     medians = {c: (statistics.median(p) if p else 0.0) for c, p in cat_prices.items()}
124
125     seen = set()
126     cleaned = []
127     for o in self.orders:
128         if o.order_id in seen:
129             continue
130         seen.add(o.order_id)
131         if o.price <= 0:
132             o.price = medians.get(o.category, 0.0)
133         if o.total <= 0:
134             o.total = round(o.price * o.quantity, 2)
135         cleaned.append(o)
136     self.orders = cleaned
137
138     # update stock for completed orders
139     for o in self.orders:
140         if o.status == "completed":
141             self.product_stock[o.product_id] = self.product_stock.get(o.product_id, 0) - o.quantity

```

```

142
143 def aggregate_sales_by_category(self) -> Dict[str, Dict[str, float]]:
144     """Aggregate sales and quantities by category."""
145     agg: Dict[str, Dict[str, float]] = {}
146     for o in self.orders:
147         a = agg.setdefault(o.category, {"revenue": 0.0, "quantity": 0, "orders": 0})
148         a["revenue"] += o.total if o.status == "completed" else 0.0
149         a["quantity"] += o.quantity if o.status == "completed" else 0
150         a["orders"] += 1
151     return agg
152
153 def top_customers(self, top_n: int = 5) -> List[Tuple[str, float]]:
154     """Return top customers by revenue."""
155     rev: Dict[str, float] = {}
156     for o in self.orders:
157         if o.status == "completed":
158             rev[o.customer_id] = rev.get(o.customer_id, 0.0) + o.total
159     return sorted(rev.items(), key=lambda x: x[1], reverse=True)[:top_n]
160
161 def detect_fraud(self, refund_rate_threshold: float = 0.3, large_order_multiplier: float = 5.0) -> Dict[str, List]:
162     """
163     Simple heuristics:
164     - customers with refund rate above threshold
165     - orders with total >> median total for category (multiplier)
166     - many orders from same customer within short period (burst)
167     """
168     fraud = {"high_refund_customers": [], "large_orders": [], "bursty_customers": []}
169
170     # refund rates per customer
171     cust_counts: Dict[str, Dict[str, int]] = {}
172     for o in self.orders:
173         c = cust_counts.setdefault(o.customer_id, {"total": 0, "refunded": 0})
174         c["total"] += 1

```



```

175         if o.status == "refunded":
176             c["refunded"] += 1
177     for cid, counts in cust_counts.items():
178         if counts["total"] > 0 and (counts["refunded"] / counts["total"]) >= refund_rate_threshold:
179             fraud["high_refund_customers"].append(cid)
180
181     # large orders compared to category median total
182     cat_totals: Dict[str, List[float]] = {}
183     for o in self.orders:
184         cat_totals.setdefault(o.category, []).append(o.total)
185     cat_medians = {c: (statistics.median(v) if v else 0.0) for c, v in cat_totals.items()}
186     for o in self.orders:
187         med = cat_medians.get(o.category, 0.0)
188         if med > 0 and o.total > med * large_order_multiplier:
189             fraud["large_orders"].append(o.order_id)
190
191     # bursty customers: many orders within 1 hour
192     orders_by_customer: Dict[str, List[datetime]] = {}
193     for o in self.orders:
194         if o.order_ts:
195             orders_by_customer.setdefault(o.customer_id, []).append(o.order_ts)
196     for cid, ts_list in orders_by_customer.items():
197         ts_list.sort()
198         # sliding window
199         for i in range(len(ts_list)):
200             j = i
201             while j < len(ts_list) and (ts_list[j] - ts_list[i]).total_seconds() <= 3600:
202                 j += 1
203             if (j - i) >= 5: # 5+ orders within 1 hour
204                 fraud["bursty_customers"].append(cid)
205                 break
206
207     return fraud
208
209     def detect_inventory_issues(self) -> List[Tuple[str, int]]:

```

```

210     """Return products with negative stock."""
211     return [(pid, stk) for pid, stk in self.product_stock.items() if stk < 0]
212
213     def summary(self) -> str:
214         agg = self.aggregate_sales_by_category()
215         lines = [f"Orders: {len(self.orders)}"]
216         for c, v in agg.items():
217             lines.append(f"{c}: orders={v['orders']}, qty={v['quantity']}, revenue={v['revenue']:.2f}")
218         inv_issues = self.detect_inventory_issues()
219         lines.append(f"Inventory issues: {len(inv_issues)}")
220         return "\n".join(lines)
221
222     def save_csv(self, path: str) -> None:
223         p = pathlib.Path(path)
224         p.parent.mkdir(parents=True, exist_ok=True)
225         with open(p, "w", newline="", encoding="utf-8") as f:
226             writer = csv.writer(f)
227             writer.writerow(["order_id", "customer_id", "product_id", "category", "quantity", "price", "order_ts", "status", "total"])
228             for o in self.orders:
229                 writer.writerow([o.order_id, o.customer_id, o.product_id, o.category, o.quantity, f"{o.price:.2f}", o.order_ts.isoformat(),
230
231
232     def demo_run():
233         proc = ECommerceProcessor.sample_data(120)
234         print("=== Raw sample statistics ===")
235         print(f"Raw orders: {len(proc.orders)}")
236
237         proc.clean()
238         print("\n=== After cleaning ===")
239         print(proc.summary())
240
241         fraud = proc.detect_fraud()
242         print(f"\nFraud findings: { {k: len(v) for k, v in fraud.items()} }")
243         inv_issues = proc.detect_inventory_issues()
244         if inv_issues:

```

```

245         print("\nExample inventory issue:", inv_issues[0])
246
247     proc.save_csv("output/cleaned_ecommerce_orders.csv")
248     print("\nSaved cleaned CSV -> output/cleaned_ecommerce_orders.csv")
249
250
251 if __name__ == "__main__":
252     demo_run()

```

Output :

```

PS C:\Users\Reshm\Desktop\AIAC> & C:/Users/Reshm/anaconda3/python.exe c:/Users/Reshm/Desktop/AIAC/test3q2.py
=== Raw sample statistics ===
Raw orders: 120

=== After cleaning ===
Orders: 120
books: orders=36, qty=118, revenue=37040.58
electronics: orders=29, qty=76, revenue=23589.91
home: orders=24, qty=68, revenue=17166.37
apparel: orders=31, qty=89, revenue=25032.62
Inventory issues: 8

Fraud findings: {'high_refund_customers': 1, 'large_orders': 3, 'bursty_customers': 0}

Example inventory issue: ('P5', -13)

Saved cleaned CSV -> output/cleaned_ecommerce_orders.csv
PS C:\Users\Reshm\Desktop\AIAC> 

```

Observation :

- Purpose: processes e-commerce orders — loads/creates orders, cleans data, updates product stock, aggregates sales, detects fraud and inventory issues, and can save cleaned CSV.
- Structure: Order dataclass + ECommerceProcessor with methods for parsing, sample data, clean, aggregate, top_customers, detect_fraud, detect_inventory_issues, summary, save_csv, and demo_run.
- Strengths: modular design, type hints, robust parsing with fallbacks, median-fill for missing prices/totals,

simple fraud heuristics (refund rate, large orders, bursts), and demo/sample-data support.

- Output: textual summary, fraud/inventory lists, and "output/cleaned_ecommerce_orders.csv" from `demo_run()`.
- Caveats: heuristics are simple and may produce false positives; timestamp parsing and random sample data may not reflect production; performance may need improvements for very large datasets.